

DocProcessor

ドキュメントを**QA**自動化のための検証可能なフィーチャーマップに変換

概要

DocProcessorは、独立した完全に疎結合のGoモジュールで、プロジェクトのドキュメントを読み込み、構造化されたフィーチャーマップを生成し、検証カバレッジを追跡します。LLMエージェントと連携してインテリジェントなフィーチャ抽出を行う設計ですが、オフライン利用のためのヒューリスティック抽出機能も備えています。

短い説明

プロジェクトに依存しないGoモジュールで、ドキュメント処理とフィーチャーマップの抽出を行います。ドキュメントを構造化されたフィーチャーマップに変換し、どの機能が検証済みかを追跡します。LLMエージェントによるインテリジェントな抽出と、オフライン時のヒューリスティック抽出の両方に対応し、QA自動化に「虚偽のない、常に現実と一致する保証」を提供します。

長い説明

すべてのソフトウェアチームが抱える共通の問題があります。それは、ドキュメントが約束する機能とテストがカバーする内容が食い違い、両者が同じ製品を指しているのか誰も確信を持って言えないという現実です。DocProcessorは、このギャップを可視化し、測定可能にするために存在します。プロジェクトのドキュメントを基に、構造化されたフィーチャーマップ（製品が実現すると主張するすべての機能を列挙し、機械可読なモデル化したもの）を生成し、それに対する検証カバレッジを追跡します。これにより、「このドキュメントに記載された機能は本当に検証されているのか？」という議論が、答えのある問いに変わります。DocProcessorは意図的に二つのモードを備えています。利用可能な場合はLLMエージェントを用いてセマンティックなフィーチャ抽出を行い、オフライン時にはヒューリスティックなパーサーにフォールバックします。そのため、モデルへの依存は一切なく、ネットワーク接続のないCIジョブや機内モードの開発者のラップトップでも同じように動作します。

アーキテクチャ的には、独立したプロジェクト非依存の完全に疎結合なGoモジュール（CONST-051(B)）として設計されています。プロジェクト固有の値は一切含まず、利用者が同等のコードベースのサブモジュールとして組み込むため、どのプロジェクトでも他者の前提を引き継ぐことなく採用できます。また、自らが他者に課す基準を自らも遵守しています。その主張は「虚偽防止の誓約」（CONST-035）と「完全自動化カバレッジのルール」（CONST-048）に縛られており、READMEに記載されたすべての機能は、単に終了コード0を返すだけでなく、実際のエンドユーザーが利用可能な動作を確認する自動テストやChallengeスクリプトによって検証されています。ユーザー向けの文字列はすべてCONST-046のi18n翻訳レイヤーを経由します。この仕組みの目的は、閉じたループを実現することです。DocProcessorは、HelixQAが完結させるQAサイクルの入力側を担います。ドキュメントからフィーチャーマップを抽出し、HelixQAが各マップされた機能を実行時のエビデンスで証明することで、ドキュメント、テスト、そして実際の動作がリリースごとに徐々に乖離することなく、強制的に収束させられます。

開発の背景

ドキュメントとテストは徐々に乖離していきます。ドキュメントが約束する機能をテストが証明せず、QAは「完了」の定義すら明確にできません。DocProcessorは、ドキュメントを機械可読なフィーチャーマップに変換することで、約束された内容に対する検証カバレッジを測定可能にします。

コンテンツ

なぜゲームチェンジャーなのか

ソフトウェアデリバリーにおける最も曖昧な問い——「リリースしたものは、私たちがリリースすると言ったものと一致しているのか？」——を、自動化可能で継続的に検証できる形に変える。しかも、特定のAIへの強い依存はなく、モデルがあればLLMによる抽出、なければヒューリスティックを用いるため、オフライン環境のランナーから完全なエージェントベースのパイプラインまで、あらゆる環境で同じ保証が得られる。

革新的なポイント

- ドキュメントから機能マップを抽出し、検証カバレッジを追跡。
- 二重抽出：LLMエージェント駆動またはヒューリスティック／オフライン対応。
- プロジェクトに依存せず、設定不要の疎結合（CONST-051(B)）。
- ブラフ防止の自己検証：READMEの主張はテスト／チャレンジによって裏付け（CONST-035/048）。

課題と解決策

- モデル非依存の動作: ヒューリスティック抽出のフォールバックにより、モジュールはオフラインでも機能。
- ドキュメントと現実の整合性維持: 構造化された機能マップと検証カバレッジの追跡により、QAループに統合。
- 再利用性: 厳密な疎結合と同一コードベースでのサブモジュール利用により解決。
- 自身の主張の信頼性: 広告された機能ごとに、ブラフ防止のテスト／チャレンジを実施。

テックスタック (理由と方法)

- **Go (1.25+)** ——モジュールのコア。Apache-2.0ライセンス。
- **LLM** エージェント——インテリジェントな意味論的機能抽出 (オプション)。
- ヒューリスティックパーサー——オフライン時の機能抽出フォールバック。
- **i18n** 翻訳ツール (**pkg/i18n**) ——CONST-046に基づくローカライズ文字列。
- チャレンジハーネス——モジュール自身の主張に対するブラフ防止検証。